Documentation de passation technique - ROBUS

Derniere mise a jour : 2026-09-21

1. Vue d'ensemble

ROBUS est une application Laravel 9 / PHP 8 qui expose :

- une interface web d'administration et de suivi operationnel ;
- une API mobile/API integration sous `/api/v1` ;
- des workflows financiers : rechargements, paiements, virements, commissions, KYC, cartes, transferts Mobile Money ;
- une integration Pixel Innov / PixPay pour les paiements Mobile Money multi-pays ;
- une integration Paystack ;
- des notifications temps reel via Pusher / broadcasting.

Le code applicatif principal est dans `app/`, les routes dans `routes/`, les vues dans `resources/views/`, les assets publics dans `public/` et les migrations dans `database/migrations/`.

2. Stack technique

- Backend : Laravel `^9.19`, PHP `^8.0.2`
- Auth API : Laravel Passport
- Frontend : Blade, Livewire 2, Vue 3, Vite, Bootstrap
- Base de donnees : MySQL
- Cache/session par defaut : fichiers
- Queue par defaut : `sync`
- Temps reel : Pusher / Laravel Echo
- PDF : `barryvdh/laravel-dompdf`
- Swagger/OpenAPI : `darkaonline/l5-swagger`
- QR Code : `endroid/qr-code`

Fichiers manifestes :

- `composer.json`
- `package.json`
- `vite.config.js`
- `phpunit.xml`

3. Installation locale

Pre-requis :

- PHP 8.0.2 ou plus compatible Laravel 9
- Composer
- Node.js/npm
- MySQL
- Extensions PHP usuelles Laravel : `pdo_mysql`, `mbstring`, `openssl`, `tokenizer`, `xml`, `ctype`, `json`, `fileinfo`

Commandes :

```
composer install
npm install
cp .env.example .env
php artisan key:generate
php artisan migrate
npm run dev
php artisan serve
```

Pour Passport, verifier si les cles existent deja dans `.env` ou via `storage/oauth-*`. Si besoin :

```
php artisan passport:keys
php artisan passport:client --personal
```

4. Variables d'environnement importantes

Ne jamais committer de secrets reels. Le fichier `.env.example` actuel contient des valeurs sensibles et doit etre nettoye avant passation externe ou publication.

Variables principales :

```
APP_NAME=ROBUS
APP_ENV=production
APP_KEY=
APP_DEBUG=false
APP_URL=

DB_CONNECTION=mysql
DB_HOST=
DB_PORT=3306
DB_DATABASE=
DB_USERNAME=
DB_PASSWORD=

MAIL_MAILER=smtp
MAIL_HOST=
MAIL_PORT=465
MAIL_USERNAME=
MAIL_PASSWORD=
MAIL_ENCRYPTION=ssl
MAIL_FROM_ADDRESS=
MAIL_FROM_NAME="ROBUS"

BROADCAST_DRIVER=pusher
PUSHER_APP_ID=
PUSHER_APP_KEY=
PUSHER_APP_SECRET=
PUSHER_APP_CLUSTER=

QUEUE_CONNECTION=sync
SESSION_DRIVER=file
SESSION_LIFETIME=30

PAYSTACK_PUBLIC_KEY=
PAYSTACK_SECRET_KEY=
PAYSTACK_CALLBACK_URL=
PAYSTACK_WEBHOOK_URL=

PIXEL_INNOV_API_KEY=
PIXEL_INNOV_API_KEY_XAF=
PIXEL_INNOV_SANDBOX=false
PIXEL_INNOV_PUBLIC_URL=
PIXEL_INNOV_WAVE_BUSINESS_NAME_ID=

COM_ZONE_CRON_SECRET=
```

Notes :

- `PIXEL_INNOV_PUBLIC_URL` doit etre une URL publique accessible par Pixel Innov pour les IPN.

- `PIXEL_INNOV_API_KEY` sert aux services hors XAF ; `PIXEL_INNOV_API_KEY_XAF` sert aux services marques `is_xaf`.
- `APP_DEBUG` doit etre `false` en production.
- Apres modification de `.env` en production : `php artisan config:clear && php artisan config:cache`.

5. Deploiement

Procedure type :

```
git pull origin main
composer install --no-dev --optimize-autoloader
npm ci
npm run build
php artisan migrate --force
php artisan config:cache
php artisan route:cache
php artisan view:cache
php artisan storage:link
```

Si l'application utilise des workers ou un service PHP-FPM, redemarrer les services apres deploiement.

Attention : des routes web utilitaires executent des commandes Artisan directement depuis HTTP dans `routes/web.php`. Les conserver uniquement si elles sont protegees par secret/IP/reseau interne.

6. Taches planifiees

Le scheduler Laravel est defini dans `app/Console/Kernel.php`.

Commandes planifiees :

- `app:close-status` toutes les minutes
- `CheckAppCompanyStatus` toutes les minutes
- `pixel:sync-pending` toutes les 5 minutes

Cron serveur attendu :

```
* * * * * cd /chemin/vers/robus && php artisan schedule:run >> /dev/null 2>&1
```

La commande `pixel:sync-pending` resynchronise les transactions Pixel Innov `PENDING` creees depuis plus de 3 minutes et moins de 24 heures.

7. Modules principaux

Administration web

Routes principales dans `routes/web.php`.

Fonctionnalites :

- authentification admin, master, compagnie, com zone ;
- gestion users, prospects, distributeurs, sous-distributeurs ;
- KYC et pieces justificatives ;
- commissions et historiques ;
- QR code avis clients ;
- reporting Pixel Mobile Zone.

API mobile / API client

Routes principales dans `routes/api.php`, prefixees par `/api/v1`.

Groupes importants :

- `/auth` : inscription, login, verification, reset password ;
- `/user` : dashboard, profil, KYC, historiques ;
- `/paystack` : paiement Paystack ;
- `/pixelInnov` : services, calculs, initialisation, statut, Mobile Zone.

Les routes sensibles sont protegees par `auth:api` et `check.suspended`.

8. Integration Pixel Innov / PixPay

Fichiers principaux :

- `app/Http/Controllers/PixelPaymentController.php`
- `app/Http/Services/PixelInnovService.php`
- `app/Models/PixelInnoveService.php`
- `app/Models/PixelMobileZoneTransfer.php`
- `app/Console/Commands/SyncPendingPixelTransactions.php`
- `config/services.php`

Configuration des services

Les services Pixel sont stockes dans `pixel_innove_services`.

Champs importants :

- `service_id` : ID du service cote Pixel Innov ;
- `country_id`, `provider_id`, `provider_type_id` ;

- `operation_type` : `Cash In` ou `Cash Out` ;
- `fixed_fee` : part frais banque/operateur ;
- `commission_percent` : part commission admin ;
- `commission_under_pct` : commission sous-distributeur ;
- `commission_companie_pct` : commission compagnie ;
- `status` : active/desactive le service ;
- `is_xof`, `is_xaf`, `is_cdf`, `is_gmd` : zone/devise du service.

Endpoints Pixel Innov

Sous `/api/v1/pixelInnov` :

- `GET /services` : liste les services disponibles, filtres par pays/type operation.
- `GET /zone-services` : liste les services de transfert mobile zone groupes par pays.
- `POST /calculate` : calcule frais et montant net.
- `POST /initialize` : initialise une transaction.
- `POST /status` : verifie et resynchronise une transaction.

Sous `/api/v1/pixelInnov/mobile-transfer` :

- `GET /services`
- `POST /calculate`
- `POST /initialize`
- `POST /status`

Callbacks publics dans `routes/web.php` :

- `POST /payment_pixelInnov/callback`
- `POST /payment_pixelInnov/mobile-transfer/callback`
- `GET /payment_pixelInnov/success`
- `GET /payment_pixelInnov/error`

Flux transaction simple

1. Le client appelle `/api/v1/pixelInnov/services`.
2. Il appelle `/api/v1/pixelInnov/calculate`.
3. Il appelle `/api/v1/pixelInnov/initialize`.
4. L'application cree un `WalletReload` en `PENDING`.
5. L'application appelle Pixel Innov `/transaction/airtime`.
6. Pixel Innov envoie un IPN sur `/payment_pixelInnov/callback`.
7. Le hash IPN est verifie avec `SHA256(api_key + transaction_id)`.
8. Selon l'etat :

- `SUCCESS` / `SUCCESSFUL` : credit/debit wallet, commissions, `PaymentReload`, `LedgerEntry`.

- `FAILED` / `ERROR` / `CANCELLED` : statut `ERROR`, remboursement si wallet debite a l'initialisation.

Important : le code utilise `operation_type` de Pixel Innov avec des commentaires historiques parfois inverses. Se fier au comportement actuel du code :

- `operation_type === 'Cash In'` : le wallet ROBUS est debite a l'initialisation, puis l'IPN confirme l'envoi vers Mobile Money.
- `operation_type === 'Cash Out'` : Pixel debite le mobile et le wallet ROBUS est credite apres IPN success.

Flux Mobile Zone

Le Mobile Zone fait deux jambes :

1. Cash Out expéditeur : debit du mobile de l'expediteur.
2. Cash In destinataire : envoi au telephone du destinataire.

Table : `pixel_mobile_zone_transfers`.

Statuts :

- `sender_status` : statut jambe expediteur.
- `recipient_status` : statut jambe destinataire.
- `status` : statut global `PENDING`, `PROCESSING`, `SUCCESS`, `FAILED`, `REFUNDED`.
- `refund_transaction_id`, `refund_status`, `refunded_at` : remboursement automatique si la jambe destinataire echoue apres debit expediteur.

La commission est creditee dans `creditMobileTransferCommission()` quand les deux jambes sont `SUCCESS`.

9. Base de donnees - tables sensibles

Tables metier frequentes :

- `sid_users` : comptes clients, admins, companies, sous-distributeurs.
- `wallets` : soldes utilisateurs.
- `wallet_reloads` : rechargements et transactions Pixel simples.
- `payment_reloads` : journal de paiements/rechargements finalises.
- `wallet_commissions` : wallet/pool de commission.
- `wallet_admins` : solde admin.
- `ledger_entries` : ecritures comptables.
- `pixel_innove_services` : catalogue Pixel Innov.
- `pixel_mobile_zone_transfers` : transferts Mobile Zone.
- `providers`, `provider_types`, `countries`, `currencies` : referentiels.

Avant toute intervention en production sur soldes/commissions, faire une sauvegarde DB et noter les IDs des lignes concernees.

10. Logs et diagnostic

Logs Laravel :

- `storage/logs/laravel.log`
- selon configuration, logs journaliers si `LOG_CHANNEL=daily`

Recherches utiles :

```
grep -n "Pixel Innov" storage/logs/laravel.log
grep -n "Mobile Zone Transfer" storage/logs/laravel.log
php artisan pixel:sync-pending
php artisan route:list | grep pixelInnov
```

Verifier une transaction Pixel :

- transaction simple : `wallet_reloads.reference = transaction_id`
- Mobile Zone : `pixel_mobile_zone_transfers.sender_transaction_id`, `recipient_transaction_id` ou `refund_transaction_id`

11. Tests et qualite

Tests presents :

- `tests/Unit/LedgerEntryTest.php`
- `tests/Unit/OperatorServiceCommissionTest.php`
- tests Laravel exemples

Commandes :

```
php artisan test
vendor/bin/phpunit
npm run build
```

Avant livraison d'une modification financiere :

- tester le calcul de frais ;
- tester l'initialisation ;
- tester IPN success ;
- tester IPN failed ;
- tester idempotence avec IPN duplique ;
- tester `pixel:sync-pending` si IPN manque.

12. Runbook incidents

Transaction Pixel reste PENDING

1. Chercher `wallet_reloads.reference`.
2. Appeler `/api/v1/pixelInnov/status` avec `transaction_id`.
3. Lancer `php artisan pixel:sync-pending`.
4. Verifier les logs `Pixel Innov - statut synchronise manuellement`.

IPN refuse

Verifier :

- `PIXEL_INNOV_PUBLIC_URL` public et correct ;
- callback accessible en POST ;
- `PIXEL_INNOV_API_KEY` et `PIXEL_INNOV_API_KEY_XAF` correctes ;
- hash recu vs hash attendu ;
- service `is_xaf` correct pour choisir la bonne cle.

Mobile Zone bloque

1. Chercher la ligne dans `pixel_mobile_zone_transfers`.
2. Si `sender_status=SUCCESS` et `recipient_transaction_id` vide, appeler `/api/v1/pixelInnov/mobile-transfer/status`.
3. Si `recipient_status=FAILED`, verifier `refund_transaction_id` et `refund_status`.
4. Ne pas relancer manuellement un Cash In sans verifier les transactions Pixel distantes.

Solde ou commission incoherent

1. Identifier la transaction source (`wallet_reloads` ou `pixel_mobile_zone_transfers`).
2. Comparer `payment_reloads`, `ledger_entries`, `wallets`, `wallet_admins`, `wallet_commissions`.
3. Verifier si l'IPN a ete traite plusieurs fois ou si une synchronisation manuelle a ete lancee.
4. Toute correction SQL doit etre documentee avec backup, ancien solde, nouveau solde, motif et operateur.

13. Securite - priorites de passation

Actions recommandees immediates :

- supprimer les secrets de `.env.example` et regenerer ceux qui ont ete exposes ;
- verifier que `.env` n'est pas suivi par Git ;
- desactiver `APP_DEBUG` en production ;
- proteger ou supprimer les routes web de maintenance qui executent des commandes Artisan ;

- verifier les permissions de `storage/` et `bootstrap/cache/` ;
- limiter l'acces DB production aux IPs necessaires ;
- conserver les cles Passport hors depot ;
- auditer les routes publiques de callback et cron.

14. Checklist de passation

- Acces serveur, domaine, SSL et fournisseur DNS.
- Acces base MySQL et procedure de backup/restauration.
- Acces Pixel Innov / PixPay : cles XOF et XAF, sandbox/prod, documentation operateur.
- Acces Paystack.
- Acces SMTP Infomaniak ou autre fournisseur mail.
- Acces Pusher.
- Acces depot Git et branche de production.
- Procedure de deploiement validee.
- Cron Laravel actif.
- Liste des contacts support : hebergeur, Pixel Innov, Paystack, SMTP, responsable metier.
- Export recent de la base et localisation des sauvegardes.
- Documentation des comptes admin et roles.

15. Fichiers a connaitre

- `routes/api.php` : API mobile/client.
- `routes/web.php` : interface web, callbacks paiement et routes utilitaires.
- `app/Http/Controllers/PixelPaymentController.php` : integration Pixel Innov.
- `app/Http/Services/PixelInnovService.php` : client HTTP Pixel Innov.
- `config/services.php` : configuration services tiers.
- `app/Console/Kernel.php` : scheduler.
- `app/Console/Commands/SyncPendingPixelTransactions.php` : recuperation transactions Pixel.
- `app/Models/LedgerEntry.php` : journal comptable.
- `app/Models/PixelInnoveService.php` : services Pixel Innov.
- `app/Models/PixelMobileZoneTransfer.php` : Mobile Zone.